PHÂN HIỆU TRƯỜNG ĐẠI HỌC FPT TẠI TPHCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN

BÁO CÁO MÔN HỌC

Đề tài: "VisiBike - Hệ thống Đèn thông minh cho Xe đạp"

Thành viên:

Lâm Duy Lộc (SE190907)

Vũ Quốc Anh (SE203519)

Trương Đông Hưng (SE201121)

Lê Nguyễn Huy Hoàng (SE201132)

TP. HỒ CHÍ MINH – 11/2025

MỤC LỤC

DANH MỤC HÌNH v
DANH MỤC HÌNH ..... **ERROR! BOOKMARK NOT DEFINED.**

CHƯƠNG 1 1

GIỚI THIỆU 1

1.1 GIỚI THIỆU 1
1.2 NHIỆM VỤ ĐỒ ÁN 1

CHƯƠNG 2 3

CƠ SỞ LÝ THUYẾT 3

2.1 BOARD PHÁT TRIỂN NUCLEO F429ZI 3
2.2 LED 3 WATT CÔNG SUẤT CAO 4
2.3 MẠCH ĐIỀU KHIỂN CÔNG SUẤT (SPARKFUN MOTOR DRIVER - TB6612FNG) 5

CHƯƠNG 3 6

THIẾT KẾ HỆ THỐNG 6

3.1 THIẾT KẾ PHẦN CỨNG 6
3.2 LƯU ĐỒ THUẬT TOÁN 7
3.3 PHÂN TÍCH MÃ NGUỒN 11

CHƯƠNG 4 15

KẾT QUẢ 15

4.1 KẾT QUẢ ĐẠT ĐƯỢC 15
4.1 HÌNH ẢNH SẢN PHẨM 15

TÀI LIỆU THAM KHẢO 16

PHỤ LỤC A : MÃ NGUỒN HỆ THỐNG 16

DANH MỤC HÌNH

Hình 2.1: Vi điều khiển Nucleo F411RE

Hình 2.2: Led 3-Watt công suất cao.

Hình 2.3: Sparkfun motor driver-TB6612FNG.

Hình 3.1: Sơ đồ khối hệ thống "Visi-Bike"

Hình 3.2: Lưu đồ thuật toán của hệ thống

Hình 4.1: Sản phẩm thực tế.

CHƯƠNG 1
GIỚI THIỆU

1.1 GIỚI THIỆU

Trong những năm gần đây, xe đạp ngày càng được ưa chuộng như một phương tiện di chuyển "xanh", thân thiện với môi trường và rèn luyện sức khỏe. Tuy nhiên, việc tham gia giao thông bằng xe đạp, đặc biệt là trong môi trường đô thị đông đúc hoặc vào ban đêm, tiềm ẩn nhiều rủi ro.

Một trong những nguyên nhân chính là khả năng nhận diện (visibility) của xe đạp rất thấp. Xe đạp truyền thống không được trang bị hệ thống đèn báo hiệu (xi-nhan) và đèn cảnh báo (đèn phanh). Điều này khiến các phương tiện khác khó khăn trong việc quan sát và phán đoán hướng di chuyển của người đi xe đạp, dẫn đến nguy cơ tai nạn cao.

Các giải pháp hiện có trên thị trường thường là các đèn LED pin rời, cường độ sáng yếu, và quan trọng là không tích hợp các chức năng báo hiệu thông minh như đèn xi-nhan. Nhận thấy nhu cầu cấp thiết này, chúng tôi thực hiện đề tài "VisiBike: Hệ thống Đèn thông minh cho Xe đạp" nhằm giải quyết vấn đề an toàn cho người đi xe đạp.

1.2 NHIỆM VỤ ĐỒ ÁN

1.2.1 Mục tiêu

Thiết kế và chế tạo một hệ thống đèn điện tử thông minh, nhỏ gọn, dễ dàng lắp đặt trên bất kỳ xe đạp nào, nhằm tăng cường khả năng nhận diện và báo hiệu.

1.2.2 Nhiệm vụ cụ thể

- **Nghiên cứu và lựa chọn linh kiện:**
  - Chọn bộ xử lý trung tâm: **Board Nucleo F429ZI**.
  - Chọn thiết bị chiếu sáng: **LED 3-Watt** công suất cao.
  - Chọn thiết bị đầu vào: **Nút nhấn (Pushbuttons)**.
  - Chọn giải pháp điều khiển công suất: **SparkFun Motor Driver (Dual TB6612FNG)**.
  - Chọn nguồn điện: **Pin 9/6V (Battery Pack)**.

- **Thiết kế và thi công phần cứng:**
  - Giải quyết vấn đề kỹ thuật cốt lõi: Điều khiển LED công suất cao (yêu cầu dòng lớn) bằng tín hiệu logic mức thấp (từ Nucleo).
  - Lắp ráp hệ thống trên **BreadBoard** để thử nghiệm.
- **Thiết kế và lập trình phần mềm:**
  - Lập trình cho board Nucleo để đọc tín hiệu từ 4 nút nhấn.
  - Lập trình logic điều khiển 3 LED 3W với các chức năng: Xi-nhan trái (nhấp nháy), Xi-nhan phải (nhấp nháy), và Đèn phanh (Bật/Tắt).

CHƯƠNG 2
CƠ SỞ LÝ THUYẾT

2.1 BOARD PHÁT TRIỂN NUCLEO F429ZI

Board STM32 Nucleo-144 (NUCLEO-F429ZI) là một board phát triển dựa trên vi điều khiển STM32F429ZIT6.

Tính năng: Lõi ARM Cortex-M4, xung nhịp 180 MHz, hỗ trợ lập trình và nạp code dễ dàng qua cổng USB (ST-LINK/V2-1).

Lý do lựa chọn: Board cung cấp số lượng chân I/O dồi dào, khả năng xử lý mạnh mẽ, cộng đồng hỗ trợ lớn và tương thích với nhiều môi trường lập trình (Mbed, CubeMX, Arduino...). Điều này rất phù hợp cho giai đoạn phát triển và thử nghiệm (prototype) sản phẩm.

Hình 2.1: Vi điều khiển Nucleo F429ZI

2.2 LED 3-WATT CÔNG SUẤT CAO

Đặc tính: Đây là loại LED siêu sáng, cung cấp cường độ chiếu sáng vượt trội so với các LED 5mm thông thường.

Thông số kỹ thuật (tiêu biểu):

- Điện áp hoạt động (Forward Voltage): 3.2V - 3.4V
- Dòng điện tiêu thụ (Forward Current): ~600-700mA

Vấn đề: Dòng điện yêu cầu (~700mA) lớn hơn rất nhiều so với khả năng cấp dòng tối đa của một chân I/O trên board Nucleo (thường chỉ khoảng 25mA).

Kết luận: Không thể điều khiển LED 3W trực tiếp từ vi điều khiển. Cần một mạch "công tắc" công suất trung gian.

Hình 2.2: Led 3-Watt công suất cao.

2.3 MẠCH ĐIỀU KHIỂN CÔNG SUẤT (SPARKFUN MOTOR DRIVER - TB6612FNG)

2.3.1 Giới thiệu

TB6612FNG là một IC điều khiển động cơ DC (Motor Driver) tích hợp hai cầu H (H-Bridge). Nó được thiết kế để điều khiển hai động cơ DC một cách độc lập.

2.3.2 Lý do lựa chọn cho VisiBike

Mặc dù được thiết kế cho động cơ, nhưng về bản chất, TB6612FNG là một **bộ công tắc điện tử công suất cao** (high-current switch).

- Nó có thể nhận tín hiệu điều khiển logic (3.3V/5V) từ Nucleo.
- Nó có thể đóng/ngắt một nguồn điện công suất lớn (lên đến 1.2A, sử dụng nguồn riêng VM).

Trong đề tài này, chúng tôi **không sử dụng nó để điều khiển động cơ**, mà sử dụng nó như một **mạch đệm (buffer) công suất** để Bật/Tắt các LED 3W.

Với 2 cầu H, nó có thể điều khiển độc lập 2 kênh (ví dụ: 1 kênh cho LED trái, 1 kênh cho LED phải). (Giả sử kênh thứ 3 cho đèn phanh được điều khiển tương tự hoặc bằng một module khác).

Hình 2.3: Sparkfun motor driver-TB6612FNG.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 THIẾT KẾ PHẦN CỨNG

3.1.1 Sơ đồ khối

Sơ đồ khối mô tả luồng thông tin và điều khiển của toàn bộ hệ thống.

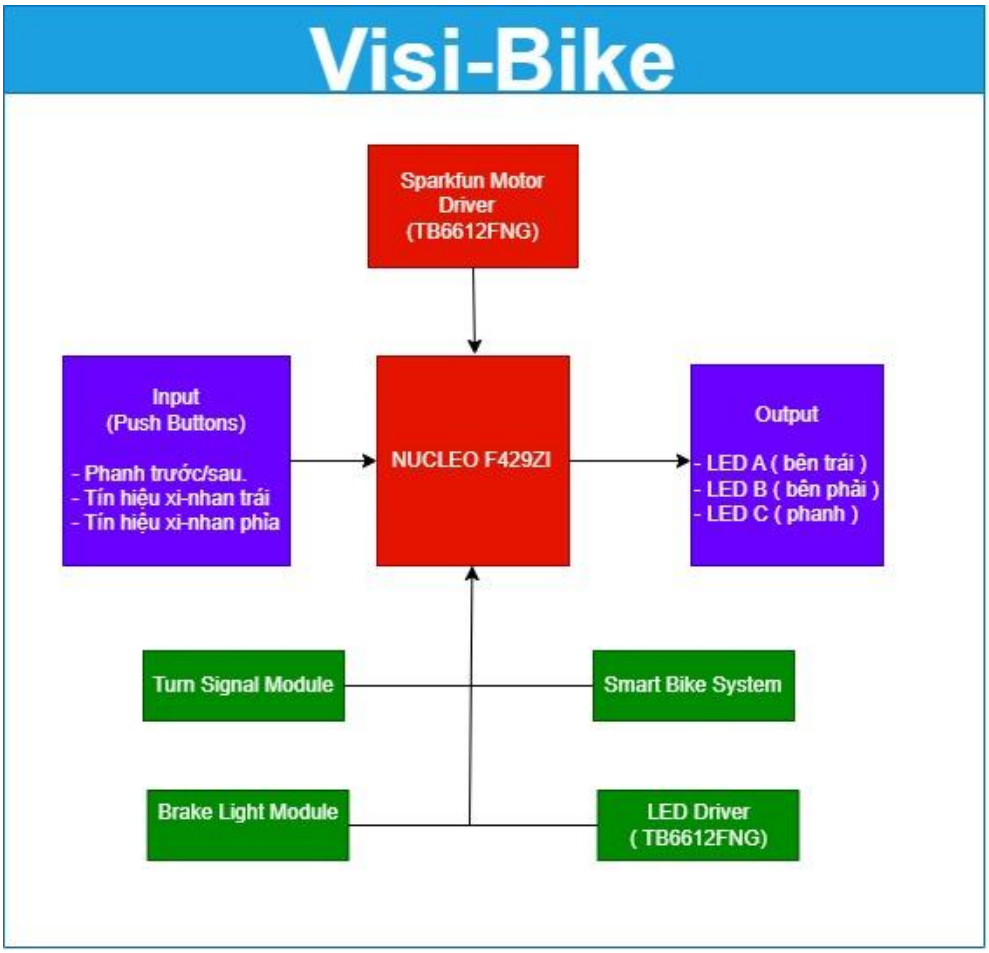
Khối Nguồn: Gồm Pin 9/6V

Khối Đầu vào: Gồm 4 Nút nhấn (Trái, Phải, Phanh) nối vào 3 chân I/O của Nucleo.

Khối Xử lý: Board Nucleo F429ZI.

Khối Điều khiển Công suất: Driver TB6612FNG.

Khối Đầu ra: 3 LED 3W.



Hình 3.1: Sơ đồ khối hệ thống "Visi-Bike"

3.2 LƯU ĐỒ THUẬT TOÁN

Diễn giải lưu đồ:

1. Bắt đầu:

- Khởi động hệ thống.

2. Khởi tạo:

- Thực thi `smartbikesystemInit()`: Cấu hình các chân I/O (nút xi-nhan trái/phải, nút phanh trước/sau), khởi tạo driver LED (TB6612FNG), đặt tất cả đèn ở trạng thái tắt (OFF).

3. Vòng lặp vô hạn (while(true)):

- Hệ thống chạy liên tục theo chu kỳ thời gian thực. Nhánh "Yes" của vòng lặp được kích hoạt.

4. Cập nhật sự kiện Xi-nhan (Bước 1 của `turnSignalUpdate`):

- Đọc và khử nhiễu nút xi-nhan trái → `leftEvent = debounceLeftInput()`
- Đọc và khử nhiễu nút xi-nhan phải → `rightEvent = debounceRightInput()`
- Kiểm tra `leftEvent == true`?
  - **Yes:** `leftBlinkerActive =! leftBlinkerActive`, `rightBlinkerActive = OFF`
  - **No:** Bỏ qua.
- Kiểm tra `rightEvent == true`?
  - **Yes:** `rightBlinkerActive =! rightBlinkerActive`, `leftBlinkerActive = OFF`
  - **No:** Bỏ qua.
- Cập nhật biến thời gian: `timeElap += TIME_INCREMENT_MS`

5. Xử lý logic Nháy đèn (Bước 2 của `turnSignalUpdate`):

- Kiểm tra `leftBlinkerActive == true`?
  - **Yes:** `timeElap = toggleLeftBlinker(timeElap)` (Hàm này sẽ gán `leftSignalState` bên trong).
  - **No:** Chuyển sang kiểm tra `rightBlinkerActive`.
- Kiểm tra `rightBlinkerActive == true`?
  - **Yes:** `timeElap = toggleRightBlinker(timeElap)` (Hàm này sẽ gán `rightSignalState` bên trong).

- **No:** (Cả hai xi-nhan đều tắt) `ledDriverSetLeftSignal(OFF)`, `ledDriverSetRightSignal(OFF)`, `timeElap = 0`.

6. Đọc Cảm biến Phanh (brakeLightUpdate):

- Tính toán logic phanh: `brakePressed =! frontBrakeButton ||! rearBrakeButton` (Do nút bấm dùng PullUp, nhấn = LOW (0), nên `!0` là `true`).

7. Cập nhật Trạng thái Phanh (ledDriverSetBrake):

- Gọi hàm `ledDriverSetBrake(brakePressed)` để gán giá trị `brakePressed` cho biến toàn cục `brakeState`.

8. Quyết định Đèn sáng (ledDriverUpdate - Logic ưu tiên):

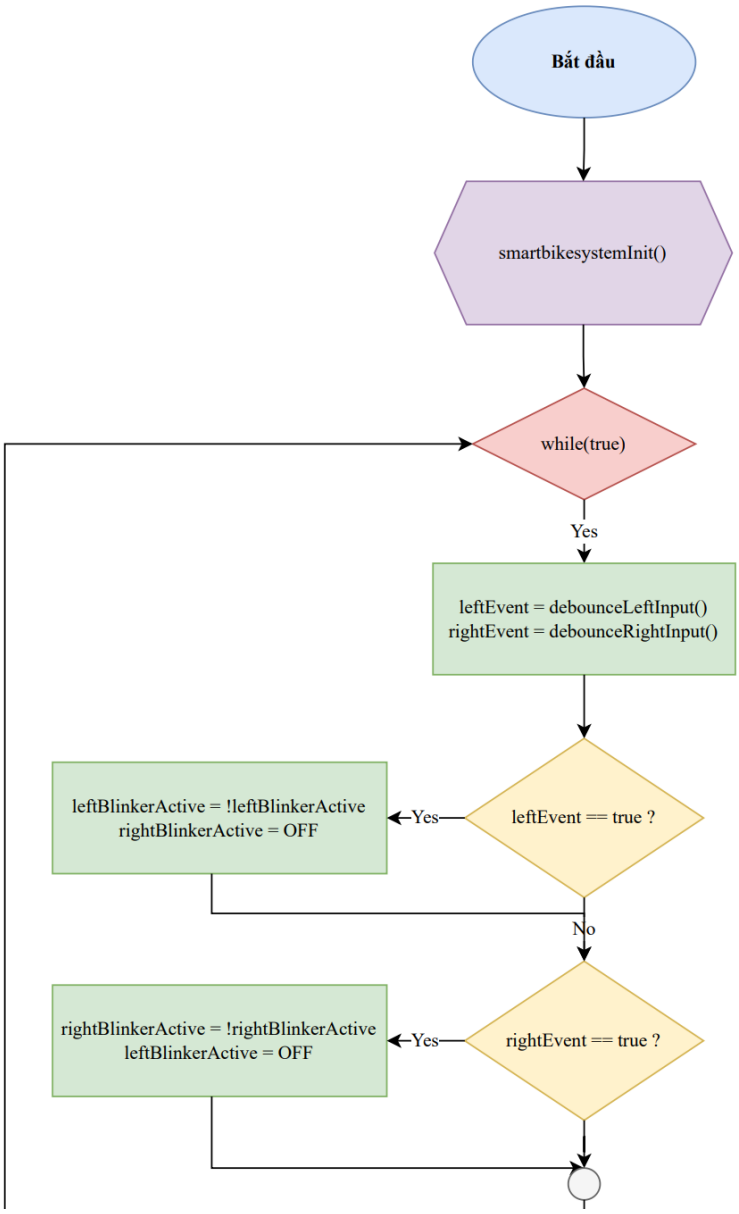
- Kiểm tra `brakeState == true`?
  - **Yes (Đang phanh):**
    - `setChannelA(OFF)` (Tắt xi-nhan trái)
    - `setChannelB(OFF)` (Tắt xi-nhan phải)
    - `ledC = ON` (Bật đèn phanh)
  - **No (Không phanh):**
    - `ledC = OFF` (Tắt đèn phanh)
    - `setChannelA(leftSignalState)` (Điều khiển xi-nhan trái theo trạng thái nháy)
    - `setChannelB(rightSignalState)` (Điều khiển xi-nhan phải theo trạng thái nháy)

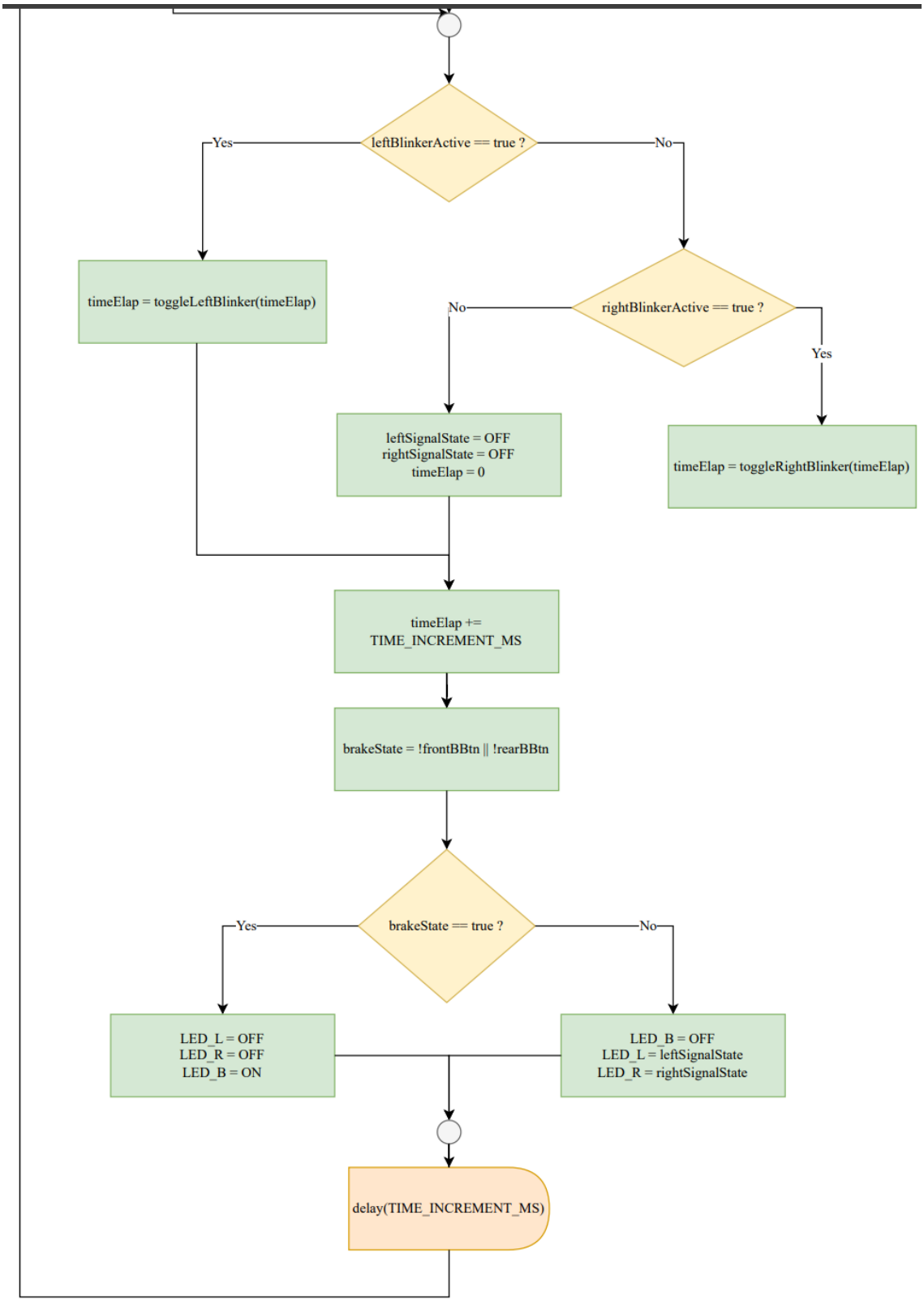
9. Chờ Đồng bộ Thời gian:

- Thực hiện `delay (TIME_INCREMENT_MS)` (ví dụ: 10ms) để đảm bảo chu kỳ xử lý ổn định.

10. Lặp lại:

- Luồng thực thi quay trở lại Bước 3 (Vòng lặp `while(true)`) để bắt đầu một chu trình cập nhật mới.


Bắt đầu
smartbikesystemInit()
while(true)
Yes
leftEvent = debounceLeftInput()
rightEvent = debounceRightInput()
leftEvent == true ?
Yes
leftBlinkerActive = !leftBlinkerActive
rightBlinkerActive = OFF
No
rightEvent == true ?
Yes
rightBlinkerActive = !rightBlinkerActive
leftBlinkerActive = OFF



Hình 3.2: Lưu đồ thuật toán.

3.3 PHÂN TÍCH MÃ NGUỒN

Mã nguồn đầy đủ được cung cấp tại Phụ lục A. Dưới đây là phân tích các phần quan trọng (đã được điều chỉnh logic cho đúng):

Khai báo & Khởi tạo (Declarations & Init)

Trước khi `main()` chạy, các đối tượng đại diện cho phần cứng được khởi tạo. Các biến trạng thái toàn cục cũng được thiết lập.

Khai báo Chân (Pin Declarations)

Các chân I/O được định nghĩa và gán cho các đối tượng:

(Từ `led_driver.cpp`)

```
DigitalOut tb_ain1(D2);
DigitalOut tb_ain2(D3);
DigitalOut tb_bin1(D5);
DigitalOut tb_bin2(D6);
DigitalOut tb_stby(D8);
DigitalOut ledC(D9);
```

(Từ `brake_light.cpp`)

```
DigitalIn frontBrakeButton(PC_8);
DigitalIn rearBrakeButton(PC_9);
```

(Từ `turn_signal.cpp`)

```
DigitalIn leftTurnSignalButton(PG_1);
DigitalIn rightTurnSignalButton(PF_9);
```

Khai báo Biến Trạng thái (State Variables)

Các biến này dùng để giao tiếp logic giữa các mô-đun:

(Từ `led_driver.cpp`)

```
static bool brakeState = false;
static bool leftSignalState = false;
static bool rightSignalState = false;
```

(Từ `turn_signal.cpp`)

```
bool leftBlinkerActive = OFF;
bool rightBlinkerActive = OFF;
```

Logic Khởi tạo (`smartbikesystemInit()`)

Hàm `main()` gọi `smartbikesystemInit()` một lần duy nhất.

```
void smartbikesystemInit(){
    turnSignalInit();  // Cài đặt 2 nút xi-nhan -> PullUp
    ledDriverInit();   // Bật tb_stby = 1; Tắt tất cả đèn
```

```
    brakeLightInit();  // Cài đặt 2 nút phanh -> PullUp
}
```

Logic chính trong `while(true)`

Hàm `main()` sau đó đi vào vòng lặp vô tận, liên tục gọi

```
smartbikesystemUpdate().
// (Trong main.cpp)
while (true) {
    smartbikesystemUpdate();
    delay(TIME_INCREMENT_MS);
}
```

Bên trong `smartbikesystemUpdate()`, các hàm con được gọi theo thứ tự ưu tiên:

1. `turnSignalUpdate()`
2. `brakeLightUpdate()`
3. `ledDriverUpdate()`

Phân tích `turnSignalUpdate()` (Đầu vào Xi-nhan)

Đây là nơi xử lý logic kích hoạt và nhấp nháy.

Đọc và Quyết định Trạng thái (Debounce & State Toggle)

Hệ thống đọc các sự kiện nhấn nút "sạch" (đã debounce).

```
// (Trong turn_signal.cpp -> turnSignalUpdate())

// Đọc sự kiện (giả sử debounceLeftInput() trả về true nếu nút vừa
được nhấn)
bool leftEvent = debounceLeftInput();
bool rightEvent = debounceRightInput();

// Logic 1: Nếu nhấn nút trái
if (leftEvent) {
    leftBlinkerActive =! leftBlinkerActive; // Đảo trạng thái (ON <->
OFF)
    rightBlinkerActive = OFF; // Tắt xi-nhan phải
}

// Logic 2: Nếu nhấn nút phải (chạy tuần tự, không phải 'else if')
if (rightEvent) {
    rightBlinkerActive =! rightBlinkerActive; // Đảo trạng thái
    leftBlinkerActive = OFF; // Tắt xi-nhan trái
}
```

Xử lý Nháy đèn (Blinking Logic)

Dựa trên trạng thái `...BlinkerActive` ở trên, hệ thống quyết định trạng thái `...SignalState` (đèn thực tế).

```
// (Trong turn_signal.cpp -> turnSignalUpdate())

if (leftBlinkerActive) {
    // Gọi hàm toggleLeftBlinker(), hàm này sẽ:
    // 1. Kiểm tra timeElap so với BLINK_TIME
    // 2. Quyết định bật hay tắt đèn
    // 3. Gọi ledDriverSetLeftSignal(ON) hoặc
ledDriverSetLeftSignal(OFF)
    timeElap = toggleLeftBlinker(timeElap);

} else if (rightBlinkerActive) {
    // Tương tự, gọi toggleRightBlinker()
    timeElap = toggleRightBlinker(timeElap);

} else {
    // Nếu cả hai hệ thống xi-nhan đều TẮT
    ledDriverSetLeftSignal(OFF);
    ledDriverSetRightSignal(OFF);
    timeElap = 0;
}

timeElap += TIME_INCREMENT_MS;
```

Phân tích `brakeLightUpdate()` (Đầu vào Phanh)

Hàm này đọc các nút phanh và gửi yêu cầu đến `led_driver`.

```
// (Trong brake_light.cpp -> brakeLightUpdate())

// Logic then chốt: Nút PullUp, nhấn = LOW (0).
// Nếu một trong hai nút là LOW (0), thì !0 = true.
bool brakePressed = !frontBrakeButton || !rearBrakeButton;

// Gửi "yêu cầu" đến led_driver
ledDriverSetBrake(brakePressed);
```

Phân tích `ledDriverUpdate()` (Điều khiển Đầu ra & Ưu tiên)

Đây là hàm cuối cùng trong chu kỳ, chịu trách nhiệm bật/tắt đèn dựa trên logic ưu tiên: phanh > xi-nhan.

```
// (Trong led_driver.cpp -> ledDriverUpdate())

// Logic ưu tiên
if (brakeState) {
    // TRƯỜNG HỢP 1: ĐANG PHANH (brakeState == true)
    // Ưu tiên phanh. Tắt tất cả xi-nhan.
    setChannelA(OFF); // Tắt LED A (Trái)
    setChannelB(OFF); // Tắt LED B (Phải)
    ledC = ON;        // Bật LED C (Phanh)

} else {
    // TRƯỜNG HỢP 2: KHÔNG PHANH
    // Chạy xi-nhan bình thường
    ledC = OFF;       // Tắt LED C (Phanh)

    // Bật/tắt đèn xi-nhan dựa trên trạng thái nháy
    // được tính toán từ turn_signal.cpp
    setChannelA(leftSignalState);
    setChannelB(rightSignalState);
}
```

CHƯƠNG 4
KẾT QUẢ

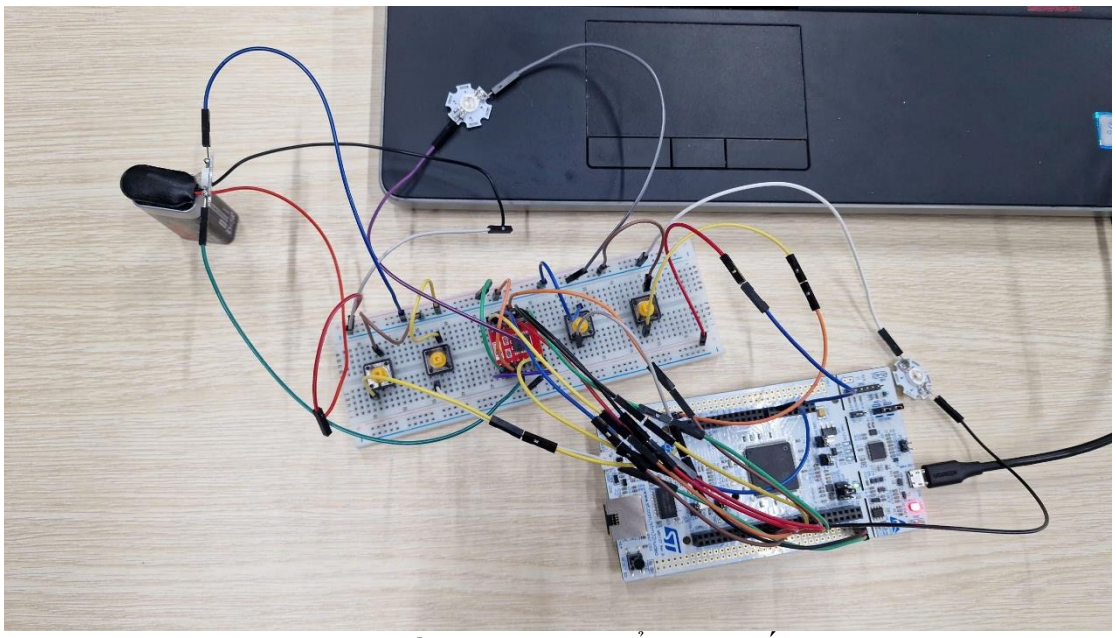
4.1 KẾT QUẢ ĐẠT ĐƯỢC

Dự án đã hoàn thành mục tiêu tổng quát (Mục 1.2.1) là thiết kế và chế tạo thành công một mô hình (prototype) hệ thống đèn điện tử thông minh, có khả năng tăng cường nhận diện và báo hiệu cho xe đạp.

Các nhiệm vụ cụ thể (Mục 1.2.2) cũng đã được hoàn thành:

1. **Về lựa chọn linh kiện:** Đã lựa chọn và sử dụng thành công board Nucleo F429ZI làm bộ xử lý trung tâm, các đèn LED 3W công suất cao, và driver TB6612FNG làm giải pháp điều khiển công suất.
2. **Về thiết kế phần cứng:** Đã giải quyết được vấn đề kỹ thuật cốt lõi là điều khiển LED công suất cao (yêu cầu dòng ~700mA) bằng tín hiệu logic mức thấp (từ Nucleo) thông qua việc ứng dụng driver TB6612FNG như một công tắc điện tử.
3. **Về thiết kế phần mềm:** Đã lập trình thành công logic điều khiển cho 4 nút nhấn, quản lý các trạng thái xi-nhan (nhấp nháy) và phanh, đồng thời triển khai logic ưu tiên (phanh > xi-nhan) một cách chính xác.

4.1 HÌNH ẢNH SẢN PHẨM



Hình 4.1: Sản phẩm thực tế.

TÀI LIỆU THAM KHẢO

1. Trang chủ Mbed OS: https://os.mbed.com/
2. Datasheet Vi điều khiển Nucleo F429ZI.

PHỤ LỤC A: MÃ NGUỒN HỆ THỐNG

break_light.cpp:

```
#include "arm_book_lib.h"
#include "brake_light.h"
#include "led_driver.h"
DigitalIn frontBrakeButton(PC_8);
DigitalIn rearBrakeButton(PC_9);


void brakeLightInit() {
    frontBrakeButton.mode(PullUp);
    rearBrakeButton.mode(PullUp);
}

void brakeLightUpdate() {

    bool brakePressed = !frontBrakeButton || !rearBrakeButton;

    ledDriverSetBrake(brakePressed);
}

```

led_driver.cpp

```cpp
#include "mbed.h"
#include "arm_book_lib.h"
#include "smart_bike_system.h"
#include "led_driver.h"

#define TB_AIN1_PIN     D2
#define TB_AIN2_PIN     D3
#define TB_PWMA_PIN     D4
#define TB_BIN1_PIN     D5
#define TB_BIN2_PIN     D6
#define TB_PWMB_PIN     D7
#define TB_STBY_PIN     D8


#define LED_C_PIN       D9

DigitalOut tb_ain1(TB_AIN1_PIN);
DigitalOut tb_ain2(TB_AIN2_PIN);
DigitalOut tb_pwma(TB_PWMA_PIN);
DigitalOut tb_bin1(TB_BIN1_PIN);
DigitalOut tb_bin2(TB_BIN2_PIN);
DigitalOut tb_pwmb(TB_PWMB_PIN);
DigitalOut tb_stby(TB_STBY_PIN);

DigitalOut ledC(LED_C_PIN);

static bool brakeState = false;
static bool leftSignalState = false;
static bool rightSignalState = false;

static void setChannelA(bool on);

static void setChannelB(bool on);

void ledDriverInit() {

    tb_stby = 1;

    tb_pwma = 1;
    tb_pwmb = 1;

    setChannelA(OFF);
    setChannelB(OFF);
    ledC = OFF;
```

```
}

void ledDriverUpdate() {

    if (brakeState) {

        setChannelA(OFF);
        setChannelB(OFF);
        ledC = ON;
    }
    else {

        ledC = OFF;

        setChannelA(leftSignalState);
        setChannelB(rightSignalState);
    }
}

void ledDriverSetBrake(bool state) {
    brakeState = state;
}

void ledDriverSetLeftSignal(bool state) {
    leftSignalState = state;
}

void ledDriverSetRightSignal(bool state) {
    rightSignalState = state;
}

static void setChannelA(bool on) {
    if (on) {
        tb_ain1 = 1;
        tb_ain2 = 0;
    } else {
        tb_ain1 = 0;
        tb_ain2 = 0;
    }
}

static void setChannelB(bool on) {
    if (on) {
        tb_bin1 = 1;
        tb_bin2 = 0;
    } else {
        tb_bin1 = 0;
        tb_bin2 = 0;
    }
}
```

smart_bike_system.cpp

```cpp
#include "mbed.h"
#include "arm_book_lib.h"
#include "smart_bike_system.h"
#include "brake_light.h"
#include "turn_signal.h"
#include "led_driver.h"

void smartbikesystemInit(){
    turnSignalInit();
    ledDriverInit();
    brakeLightInit();
}

void smartbikesystemUpdate(){
    turnSignalUpdate();
    brakeLightUpdate();
    ledDriverUpdate();
 }

```

turn_signal.cpp

```cpp
#include "turn_signal.h"
#include "mbed.h"
#include "arm_book_lib.h"
#include "smart_bike_system.h"
#include "led_driver.h"

#define DEBOUNCE_TIME 30
#define BLINK_TIME 100
#define TIME_ELAP_RESET 0

typedef enum {
    BUTTON_UP,
    BUTTON_DOWN,
    BUTTON_FALLING,
    BUTTON_RISING
} buttonState_t;

DigitalIn leftTurnSignalButton(PG_1);
DigitalIn rightTurnSignalButton(PF_9);

buttonState_t buttonStateLeft = BUTTON_UP;
buttonState_t buttonStateRight = BUTTON_UP;

bool leftBlinkerActive = OFF;
bool rightBlinkerActive = OFF;

static bool debounceLeftInput();
static bool debounceRightInput();
static int toggleLeftBlinker(int timeElap);
static int toggleRightBlinker(int timeElap);

void turnSignalInit(){
    leftTurnSignalButton.mode(PullUp);
    rightTurnSignalButton.mode(PullUp);
}

void turnSignalUpdate() {
    static int timeElap = 0;

    if (debounceLeftInput()) {
        leftBlinkerActive = !leftBlinkerActive;
        if (leftBlinkerActive == ON) {
            rightBlinkerActive = OFF;
        }
```

```
    }
    if (debounceRightInput()) {
        rightBlinkerActive = !rightBlinkerActive;
        if (rightBlinkerActive == ON) {
            leftBlinkerActive = OFF;
        }
    }

    if (leftBlinkerActive) {
        timeElap = toggleLeftBlinker(timeElap);
    }
    else if (rightBlinkerActive) {
        timeElap = toggleRightBlinker(timeElap);
    }
    else {
        ledDriverSetLeftSignal(OFF);
        ledDriverSetRightSignal(OFF);
        timeElap = 0;
    }
    timeElap += TIME_INCREMENT_MS;
}

static bool debounceLeftInput(){
    bool leftButtonReleasedEvent = false;
    static int timeElap = 0;
    switch( buttonStateLeft ) {

    case BUTTON_UP:
        if( leftTurnSignalButton ) {
            buttonStateLeft = BUTTON_FALLING;
            timeElap = 0;
        }
        break;

    case BUTTON_FALLING:
        if( timeElap >= DEBOUNCE_TIME ) {
            if( leftTurnSignalButton ) {
                buttonStateLeft = BUTTON_DOWN;
            } else {
                buttonStateLeft = BUTTON_UP;
            }
        }
        timeElap = timeElap + TIME_INCREMENT_MS;
        break;
```

```
        case BUTTON_DOWN:
            if( !leftTurnSignalButton ) {
                buttonStateLeft = BUTTON_RISING;
                timeElap = 0;
            }
            break;

        case BUTTON_RISING:
            if( timeElap >= DEBOUNCE_TIME ) {
                if( !leftTurnSignalButton ) {
                    buttonStateLeft = BUTTON_UP;
                    leftButtonReleasedEvent = true;
                } else {
                    buttonStateLeft = BUTTON_DOWN;
                }
            }
            timeElap = timeElap + TIME_INCREMENT_MS;
            break;
        }
        return leftButtonReleasedEvent;
    }

    static bool debounceRightInput(){
        bool rightButtonReleasedEvent = false;
        static int timeElap = 0;
        switch( buttonStateRight ) {

        case BUTTON_UP:
            if( rightTurnSignalButton ) {
                buttonStateRight = BUTTON_FALLING;
                timeElap = 0;
            }
            break;

        case BUTTON_FALLING:
            if( timeElap >= DEBOUNCE_TIME ) {
                if( rightTurnSignalButton ) {
                    buttonStateRight = BUTTON_DOWN;
                } else {
                    buttonStateRight = BUTTON_UP;
                }
            }
            timeElap = timeElap + TIME_INCREMENT_MS;
```

```
        break;

    case BUTTON_DOWN:
        if( !rightTurnSignalButton ) {
            buttonStateRight = BUTTON_RISING;
            timeElap = 0;
        }
        break;

    case BUTTON_RISING:
        if( timeElap >= DEBOUNCE_TIME ) {
            if( !rightTurnSignalButton ) {
                buttonStateRight = BUTTON_UP;
                rightButtonReleasedEvent = true;
            } else {
                buttonStateRight = BUTTON_DOWN;
            }
        }
        timeElap = timeElap + TIME_INCREMENT_MS;
        break;
    }
    return rightButtonReleasedEvent;
}

static int toggleLeftBlinker(int timeElap) {
    static bool blinkState = OFF;
    if (blinkState == ON) {
            if (timeElap>=BLINK_TIME) {
                ledDriverSetLeftSignal(OFF);
                blinkState = !blinkState;
                return TIME_ELAP_RESET;
            } else {
                return timeElap;
            }
    }
    else {
            if (timeElap>=BLINK_TIME) {
                ledDriverSetLeftSignal(ON);
                blinkState = !blinkState;
                return TIME_ELAP_RESET;
            } else {
                return timeElap;
            }
    }
```

```
}

static int toggleRightBlinker(int timeElap) {
    static bool blinkState = OFF;
    if (blinkState == ON) {
            if (timeElap>=BLINK_TIME) {
                ledDriverSetRightSignal(OFF);
                blinkState = !blinkState;
                return TIME_ELAP_RESET;
            } else {
                return timeElap;
            }
    }
    else {
            if (timeElap>=BLINK_TIME) {
                ledDriverSetRightSignal(ON);
                blinkState = !blinkState;
                return TIME_ELAP_RESET;
            } else {
                return timeElap;
            }
    }
}
```

main.cpp

```
#include "mbed.h"
#include "smart_bike_system.h"
#include "arm_book_lib.h"

int main()
{
    smartbikesystemInit();
    while (true) {
        smartbikesystemUpdate();
        delay(TIME_INCREMENT_MS);
    }
}

```